

Guía Práctica No. 1: Complejidad Temporal de Algoritmos: Análisis de Tiempo de Ejecución

Esta guía práctica abarca, análisis de tiempo de ejecución en el peor caso. Se recomienda fuertemente la lectura del capítulo 2 de *Introduction to the Design and Analysis of Algorithms* (Levitin 2003) y la sección C.1 de *Introduction to Algorithms* (Thomas H.Cormen)

Parte de esta práctica cuenta con esquemas de programación y tests de asistencia a su resolución. Éstos pueden accederse a partir del classroom de github, para el cual es conveniente tener una cuenta en Github (<http://github.com>).

Esta práctica no tiene entrega formal. Su resolución es opcional aunque se recomienda realizar cada uno de los ejercicios propuestos.

1. Considere la clase `ArraySorter` provista en el repositorio. Implemente los algoritmos de ordenamiento `selectionSort` y `bubbleSort`. Asegúrese que todos los tests pasen (debe completar la implementación de algunos métodos auxiliares). Complete el conjunto de tests de unidad provistos, con otros adicionales, para los algoritmos implementados. Utilice el proyecto para realizar una comparación experimental del tiempo de ejecución de los algoritmos implementados.
2. Dado el siguiente algoritmo para BUBBLE-SORT: Muestre que el algoritmo corre en $O(n^2)$

```
BUBBLESORT (A)
1  for i = 1 to A.length - 1
2    for j = A.length downto i + 1
3      if A[j] < A[j-1]
4        swap A[j] and A[j-1]
```

3. Realizar el análisis de tiempo de ejecución de MergeSort en el peor caso.
4. Describa la ecuación de recurrencia y calcule el tiempo de ejecución en el peor caso del algoritmo buscar. Explique qué considera como el tamaño de la entrada, y cuál es el peor caso del algoritmo.

```
public static int buscar(int[] lista, int objetivo) {
    return buscarRecursivo(lista, objetivo, 0, lista.length - 1);
}
```

```
private static int buscarRecursivo(int[] lista, int objetivo, int izquierda, int derecha) {
    if (izquierda > derecha) {
        return -1;
    }
    int medio = izquierda + (derecha - izquierda) / 2;
    if (lista[medio] == objetivo) {
        return medio;
    }
    if (lista[medio] < objetivo) {
        return buscarRecursivo(lista, objetivo, medio + 1, derecha);
    }
}
```

```

    }
    return buscarRecursoivo(lista, objetivo, izquierda, medio - 1);
}

```

5. Analizar el tiempo de ejecución del método *hasDuplicates(int[] arr)* en el peor caso, aclarar qué considera como el tamaño del input, y cuál sería el peor caso, desarrolle la ecuación del tiempo de ejecución y determine la tasa de crecimiento.

```

/**
 * Checks if there are duplicates in the array.
 */
public static boolean hasDuplicates(int[] arr) {
    return hasDuplicates(arr, 0);
}

/**
 * Recursively checks if there are duplicated elements in the array.
 */
private static boolean hasDuplicates(int[] arr, int index) {
    if (index >= arr.length) {
        return false;
    }
    if (sequentialSearch(arr, arr[index], index + 1) != -1) {
        return true;
    }
    return hasDuplicates(arr, index + 1);
}

/**
 * Recursively searches for an element in the array.
 */
public static int sequentialSearch(int[] arr, int target, int index) {
    if (index >= arr.length) {
        return -1;
    }
    if (arr[index] == target) {
        return index;
    }
    return sequentialSearch(arr, target, index + 1);
}

```